

QUESTION:

Fundamentals of Object Oriented Programming: Object oriented paradigm, Object and

Classes, Data Abstraction and Encapsulation, Inheritance, Polymorphism, Dynamic Binding.

1. Sample Case study on class & objects CO1

Bank Account Management System:

Problem Statement: Design a system to manage bank accounts.

Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

Object Usage: Create objects for each customer's account and perform transactions like deposits, withdrawals, and transfers

Step wise execution

1. Program starts. Output: "Welcome to Bank System"

2. Object s (Status) created:

- new Account(123456, "John Doe", 1234, 50000) created
- Bank constructor called via super() -> copies accountNumber, accountHolderName, pin, amount
- overdraftLimit set to 10000

3. Object sa (savingsAccount) created:

- new Account(123456, "John Doe", 1234, 50000) created
- Bank constructor called via super()
- interestRate set to 5.0

- savingamount set to 10000

4. Menu loop starts (infinite while loop):

Displays:

1- Debit

2- Credit

3- Check Overdraft

4- Calculate Interest

5- Show Account Details

6- Check Status

7- Exit

5. User enters choice = 6

-> s.status() executes

-> checks super.amount (50000) > 0

-> Output: "Account is Active"

6. User enters choice = 1

-> prompts "Enter debit amount"

-> user enters 20000

-> s.debit(20000, "susmithaa", 1234) executes

-> pin matches (1234 == 1234)

-> debitAmount (20000) <= 100000, passes limit check

-> amount (50000) >= debitAmount (20000), passes balance check

-> amount updated: 50000 - 20000 = 30000

-> Output: "Transaction Details: susmithaa"

-> Output: "20000 debited successfully"

7. User enters choice = 2

- > prompts "Enter credit amount"
- > user enters 5000
- > s.credit(5000, "Microsoft", "Credit transaction") executes
- > Output: "Transaction Details: Credit transaction"
- > Output: "Company Name: Microsoft"
- > amount updated: $30000 + 5000 = 35000$
- > Output: "5000 credited successfully"

8. User enters choice = 4

- > prompts "Enter savings amount"
- > user enters 15000
- > NEW sa object created (resets to fresh Account with amount=50000)
- > sa.savings() executes: $\text{savingamount} = 10000 \text{ (fixed bonus)} + 15000 = 25000$
- > Output: "Savings Amount: 25000"
- > sa.calculateInterest() executes (overridden version)
- > $\text{interest} = (25000 * 5.0) / 100 = 1250.0$
- > Output: "Interest earned: 1250.0"

9. User enters choice = 5

- > sa.showAccountDetails() executes
- > calls private DisplayAccountDetails()
- > Output: "Bank Name: HDFC"
- > Output: "Branch Name: Mumbai"
- > Output: "IFSC Code: HDFC0001"
- > Output: "Transaction ID: 0"

-> Output: "Account Number: 123456"

-> Output: "Account Holder Name: John Doe"

-> Output: "Current Balance: 50000"

(Note: sa's amount is 50000, unaffected by earlier debit/credit on s)

10. User enters choice = 3

-> s.checkOverdraft() executes

-> checks super.amount (35000) < 0 -> false

-> Output: "Current balance: 35000"

11. User enters choice = 7

-> System.exit(0) called

-> Program terminates

Algorithm:

1. Start the program.

2. Define class Account with fields:

- accountNumber, accountHolderName, private pin, amount, acc_type
- Constructor initializes all fields
- getPin() returns the private pin (package-level access for subclasses)
- credit() adds an amount to balance and prints transaction/company details
- debit() verifies pin, checks transaction limit (≤ 100000) and sufficient balance before deducting

3. Define class Bank that extends Account:

- Constructor takes an existing Account object and copies its details via super()
- Adds bankName, branchName, ifscCode, transactionId, interestRate
- calculateInterest() computes interest on amount

4. Define class savingsAccount that extends Bank:

- Adds savingamount, initializes via constructor with a default transaction ID of 0
- savings() adds a fixed bonus of 10000 to savingamount
- Overrides calculateInterest() to compute interest based on savingamount instead of amount

5. Define class Status that extends Bank:

- Adds overdraftLimit
- checkOverdraft() checks if balance is negative
- status() prints whether the account is Active or Inactive based on balance
- DisplayAccountDetails() (private) prints bank + account info; exposed via showAccountDetails()

6. Define class BankSystem (contains main):

- Creates a Status object s and a savingsAccount object sa, each wrapping a new Account
- Runs an infinite loop showing a menu:
(Debit, Credit, Check Overdraft, Calculate Interest, Show Account Details, Check Status, Exit)
- Based on user's choice (via Scanner), calls the corresponding method on s or sa
- case

7 exits the program via System.exit(0)

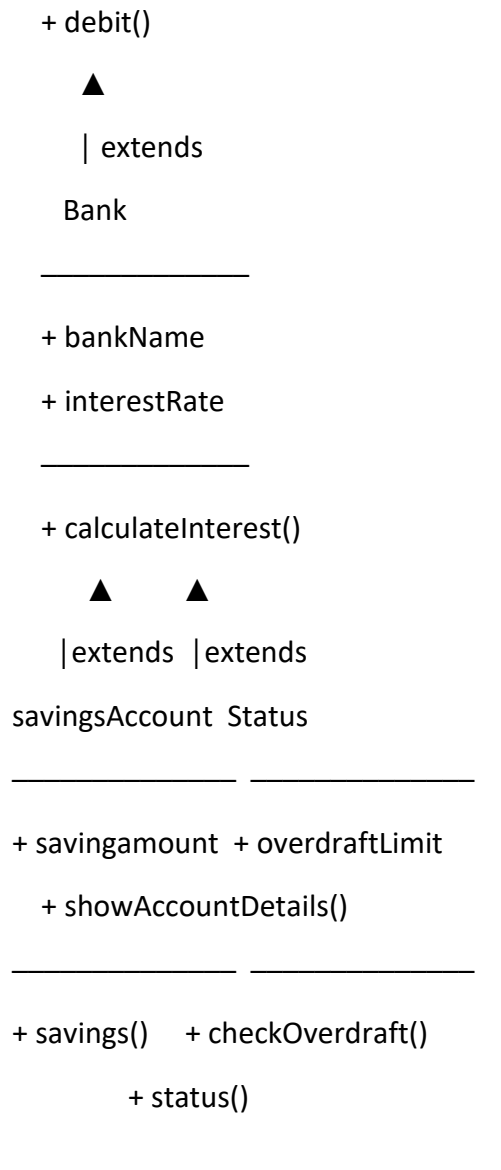
UML diagram

Account

- pin

+ amount

+ credit()



Code

```

package oops;
import java.util.Scanner;
class Account{
    int accountNumber;
    String accountHolderName;
    private int pin;
    int amount;
    String acc_type;

```

```

    public Account(int accountNumber, String accountHolderName,
int pin, int amount) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.pin = pin;
        this.amount = amount;
    }

    int getPin() {
        return pin;
    }

    int credit(int creditAmount,String companyName){
        System.out.println("Company Name: " + companyName);
        this.amount = this.amount + creditAmount;
        System.out.println(creditAmount + " credited
successfully");
        return this.amount;
    }
    int debit(int debitAmount,String personName,int pin){
        if(this.pin == pin){
            if(debitAmount > 100000){
                System.out.println("Transaction limit
exceeded");
                return this.amount;
            }else{
                if(this.amount >= debitAmount){
                    this.amount = this.amount - debitAmount;
                    System.out.println("Transaction Details: " +
personName );
                    System.out.println(debitAmount + " debited
successfully");
                }else
                    System.out.println("Insufficient balance");
            }
        } else {

```

```

        System.out.println("Invalid PIN");
    }
    return this.amount;
}
}

class Bank extends Account{
    String bankName;
    String branchName;
    String ifscCode;
    Account account;
    int trasactionId;
    double interestRate;

    public Bank(String bankName, String branchName, String
ifscCode, Account account, int trasactionId) {
        super(account.accountNumber, account.accountHolderName,
account.getPin(), account.amount);
        this.bankName = bankName;
        this.branchName = branchName;
        this.ifscCode = ifscCode;
        this.account = account;
        this.trasactionId = trasactionId;
    }

    public int getpin() {
        return getPin();
    }

    public void calculateInterest() {
        double interest = (super.amount * interestRate) / 100;
        System.out.println("Interest earned: " + interest);
    }
}

```

```
class savingsAccount extends Bank{
    int savingamount;
    public savingsAccount(int savingamount, String bankName,
String branchName, String ifscCode, Account account, double
interestRate) {
        super(bankName, branchName, ifscCode, account, 0);
        super.interestRate = interestRate;
        this.savingamount = savingamount;
    }

    void savings(){
        int savings=10000;
        savingamount=savings+savingamount;
        System.out.println("Savings Amount: " +
this.savingamount);
    }
    @Override
    public void calculateInterest() {
        double interest = (this.savingamount * interestRate) /
100;
        System.out.println("Interest earned: " + interest);
    }
}

class Status extends Bank{
    double overdraftLimit;

    public Status(String bankName, String branchName, String
ifscCode, Account account) {
        super(bankName, branchName, ifscCode, account, 0);
    }

    public void checkOverdraft() {
        if (super.amount < 0) {
            System.out.println("Overdraft limit exceeded.
Current balance: " + super.amount);
        }
    }
}
```

```

        } else {
            System.out.println("Current balance: " +
super.amount);
        }
    }

    void status(){
        if(super.amount > 0){
            System.out.println("Account is Active");
        }else{
            System.out.println("Account is Inactive");
        }
    }

    private void DisplayAccountDetails() {
        System.out.println("Bank Name: " + bankName);
        System.out.println("Branch Name: " + branchName);
        System.out.println("IFSC Code: " + ifscCode);
        System.out.println("Transaction ID: " + trasactionId);
        System.out.println("Account Number: " +
super.accountNumber);
        System.out.println("Account Holder Name: " +
super.accountHolderName);
        System.out.println("Current Balance: " + super.amount);
    }

    public void showAccountDetails() {
        DisplayAccountDetails();
    }
}

public class BankSystem {
    public static void main(String[] args) {
        System.out.println("Welcome to Bank System");
    }
}

```

```

        Status s= new Status("SBI", "Andhra Pradesh",
"SBI234C0001", new Account(123456, "Susmitha", 1234, 50000));
        Scanner sc = new Scanner(System.in);
        savingsAccount sa = new savingsAccount(10000, "SBI",
"Andhra Pradesh", "SBI234C0001", new Account(123456,
"Susmitha", 1234, 50000), 0.1800);
        while(true){
            System.out.print("1- Debit\n2- Credit\n3- Check
Overdraft\n4- Calculate Interest\n5- Show Account Details\n6-
Check Status\n7- Exit\nEnter choice: ");
            int choice = sc.nextInt();
            switch (choice) {
                case 1:
                    System.out.print("Enter debit amount: ");
                    int debitAmount=sc.nextInt();
                    s.debit(debitAmount, "susmithaa", 1234);
                    break;
                case 2:
                    System.out.print("Enter credit amount: ");
                    int creditAmount = sc.nextInt();
                    s.credit(creditAmount, "Microsoft");
                    break;
                case 3:
                    s.checkOverdraft();
                    break;
                case 4:
                    sa.savings();
                    sa.calculateInterest();
                    break;
                case 5:
                    s.showAccountDetails();
                    break;
                case 6:
                    s.status();
                    break;
                case 7:

```

```

        System.exit(0);
        break;
    default:
        System.out.println("Invalid choice");
    }
}
}
}

```

Output screen shot

```

PS C:\Users\Admin\OneDrive\Documents\basicjava> & "C:\Program Files\Java\jdk-26.0.1\bin\java.exe" "-enable-preview" "-XX:+ShowCodeDetailsInExceptionMessages" "-cp" "C:\Users\Admin\OneDrive\Documents\basicjava\bin" "oops.BankSystem"
Welcome to Bank System
1- Debit
2- Credit
3- Check Overdraft
4- Calculate Interest
5- Show Account Details
6- Check Status
7- Exit
Enter choice): 1
Enter debit amount: 2334
Transaction Details: susmithaa
2334 debited successfully
1- Debit
2- Credit
3- Check Overdraft
4- Calculate Interest
5- Show Account Details
6- Check Status
7- Exit
Enter choice): 2
Enter credit amount: 789434
Company Name: Microsoft
789434 credited successfully
1- Debit
2- Credit
3- Check Overdraft
4- Calculate Interest
5- Show Account Details
6- Check Status
7- Exit
Enter choice): 3
Current balance: 837100
1- Debit

```

A screenshot of a Windows desktop environment. The background is a dark blue gradient with a faint grid pattern. In the center, there is a white rectangular area representing a terminal or command prompt window. Inside this window, the following text is displayed:

```
File Edit Selection View Go Run Terminal Help  
basijava  
  
Problems Output Debug Console Terminal Ports  
1- Debit  
2- Credit  
3- Check Overdraft  
4- Calculate Interest  
5- Show Account Details  
6- Check Status  
7- Exit  
Enter choice: 4  
Savings Amount: 20000  
Interest earned: 36.0  
1- Debit  
2- Credit  
3- Check Overdraft  
4- Calculate Interest  
5- Show Account Details  
6- Check Status  
7- Exit  
Enter choice: 5  
Bank Name: SBI  
Branch Name: Andhra Pradesh  
IFSC Code: SBIZ3AC0001  
Transaction ID: 0  
Account Number: 123456  
Account Holder Name: Susmitha  
Current Balance: 837100  
1- Debit  
2- Credit  
3- Check Overdraft  
4- Calculate Interest  
5- Show Account Details  
6- Check Status  
7- Exit  
Enter choice: 6  
Account is Active  
1- Debit  
2- Credit  
3- Check Overdraft  
4- Calculate Interest  
5- Show Account Details  
6- Check Status  
7- Exit
```